GPU based maximum common subgraph algorithm for drug discovery application

Jayaraj. P. B
Rahamathulla. K
Gopakumar. G

Department of Computer Science
National Institute of Technology Calicut, India

May 23, 2016

Outline

1. Introduction
   - Graph, Subgraph, Clique
2. Maximum Common Subgraph(MCS) Problem
   - MCS Solution - Related Works
3. Maximum Clique Enumeration(MCE)
   - Bron-Kerbosch(BK) Algorithm for MCE
   - Parallel MCE: Related Works
4. Proposed Parallel MCE Method using GPU
5. Experimental Results
   - Serial vs GPU Experimental Results
6. Applications
   - Application1 : LIGANDMATCHER
   - Application 2 : GRAPHSCREEN
7. Conclusion
8. References

Introduction

- Graph
  - Graphs are data structures that can represent structured objects
  - eg. biomolecules, chemical compounds, social networks
  - Contains a finite set of ordered pairs called edges (or arcs) of vertices

- Subgraph
  - A subgraph of graph G is another graph with the vertex set being a subset of that of G
  - The edges of a subgraph is also a subset of that of G, restricted to the vertex set of G

Clique

- A clique in an undirected graph is a subset of its vertices such that every two vertices in the subset are connected by an edge.

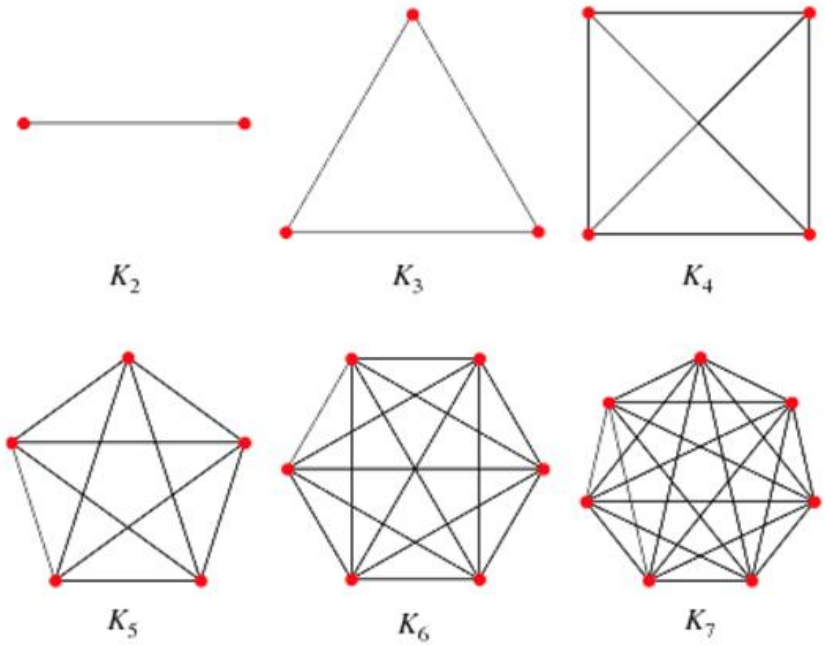


Figure: **Cliques in a Graph**

Maximum Common Subgraph

- The Maximum Common Subgraph problem(MCS)[1, 2]
  - Given two graphs G_1 and G_2, MCS problem finds the largest $H = (V, E)$, such that H is isomorphic to subgraph of G_1 and subgraph of G_2

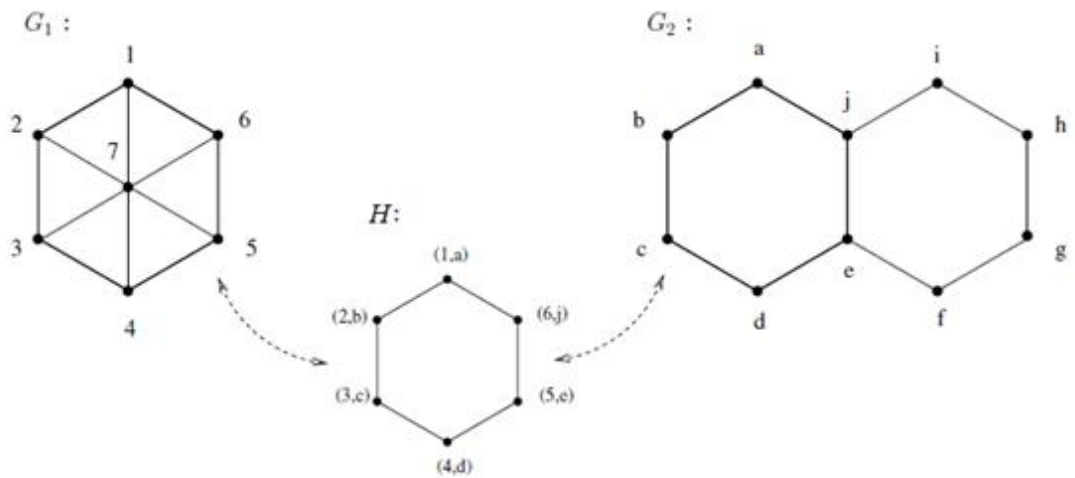


Figure: **Maximum Common Sub graph of 2 Graph**

MCS - Related Works

- Backtrack Search algorithm[3] : McGregor[4] introduced a maximal common subgraph algorithm that uses a backtrack search
- Algorithm based on Maximum Clique Enumeration(MCE): MCS of two graphs can be found by an algorithm that transforms the MCS problem into MCE[1]
- Applications[2]
  - Comparison of proteins for their secondary structures
  - Identification of changes of a chemical reaction
  - Ligand matching
  - Virtual screening

Algorithm based on Maximum Clique Enumeration

- Transformation of the MCS problem into clique problem[1]
- The compatibility information between two graphs can be stored in a new graph called **product graph**
- Compatibility graph or Edge Product Graph[5]
  - Includes the vertex set $V_H = E_1 \times E_2$
  - The edge pairs have to coincide their edge labels and vertex labels

- There is an edge between $e_H, f_H \in V_H$ with $e_H = (e_1, e_2)$ and $f_H = (f_1, f_2)$ if,
  - $e_1 \neq f_1$ and $e_2 \neq f_2$
  - e_1, f_1 in G_1 and e_2, f_2 in G_2 are connected via the same vertex
  - e_1, f_1 and e_2, f_2 are not adjacent in G_1 and G_2

Bron-Kerbosch(BK) Algorithm[6]

ENUMERATE-CLIQUES(C, P, S)
C: set of vertices belonging to current clique
P: set of vertices which can be added to C
S: set of vertices which cannot be added to C
N[u]: set of neighbors of u in G

1: Let P be the set $\{u_1, u_2, ..., u_k\}$
2: **if** $(P = \phi)$ and $(S = \phi)$ **then**
3: REPORT-CLIQUE;
4: **else**
5: **for** $i \leftarrow 1$ to k **do**
6: $P \leftarrow P \setminus \{u_i\}$
7: $P' \leftarrow P$; $S' \leftarrow S$; $N \leftarrow \{v \in V | \{u_i, v\} \in E\}$
8: ENUMERATE-CLIQUES $(C \cup \{u_i\}, P' \cap N, S' \cap N)$
9: $S \leftarrow S \cup \{u_i\}$
10: **end for**
11: **end if**

Bron-Kerbosch(BK) Algorithm Example

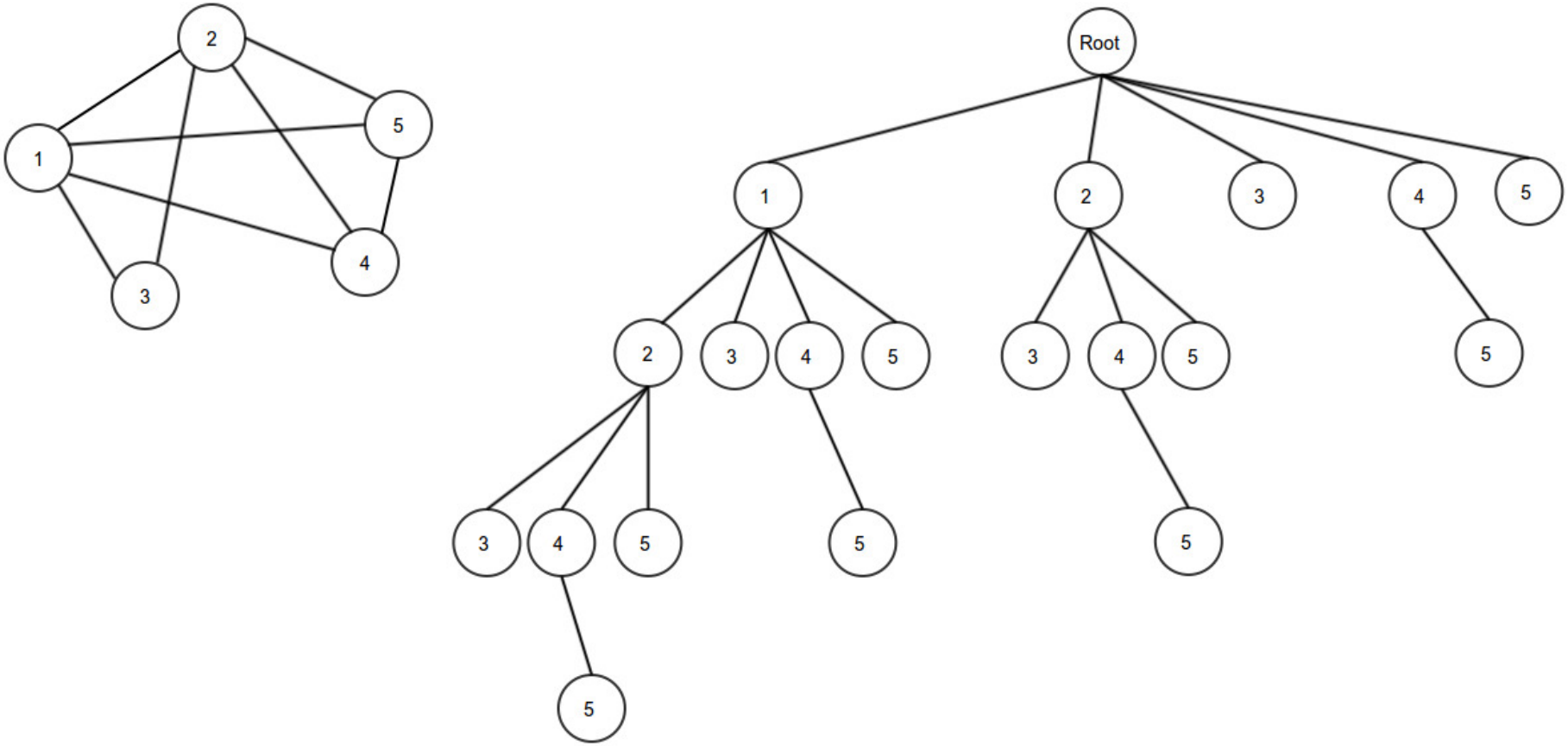


Figure: **Solution tree generated by BK algorithm**

Edge product graph

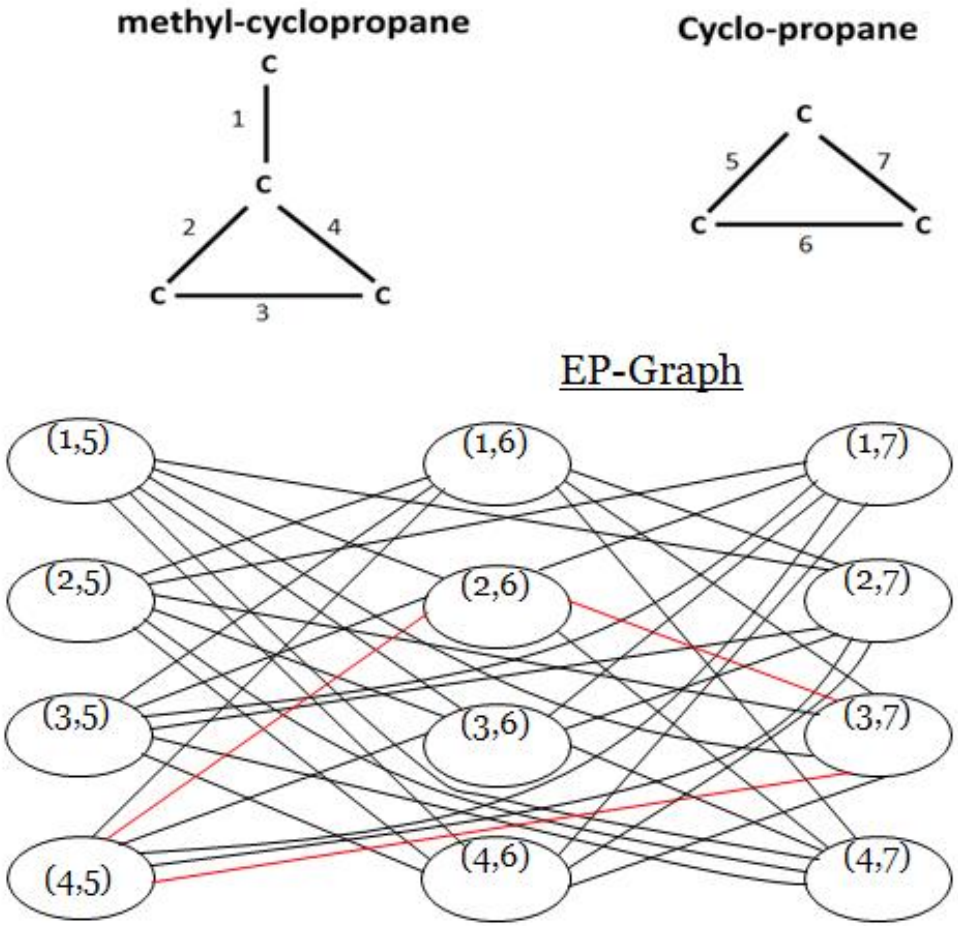


Figure: **Edge product graph of 2 given graphs**

Edge product graph

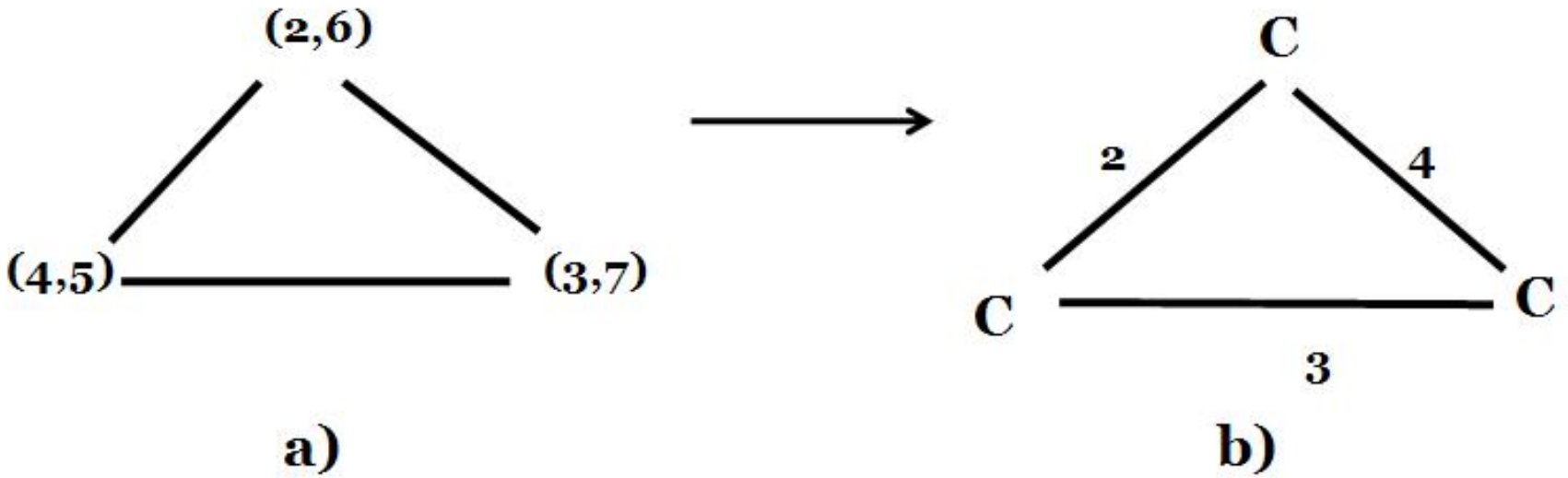


Figure: **Maximum common subgraph of the above two graphs**
a) One of the maximal clique(clique of size 3) **b)** Backtrack to form the common subgraph from the 1st graph with edges 2,4,3

Parallel BK Solution

- Clique enumeration is done by constructing a solution tree.
- For large graphs, the serial solution consumes huge amount of time for the tree construction.

- Parallel BK: Related work
  - Matthew C. Schmidt[7] proposed a parallel MCE algorithm for high performance computing architectures.
  - It involves decomposition of the search tree into search sub trees.
  - Implemented using OpenMP/POSIX thread for shared memory and MPI for distributed memory.
  - Tested on Cray XT4 machine.

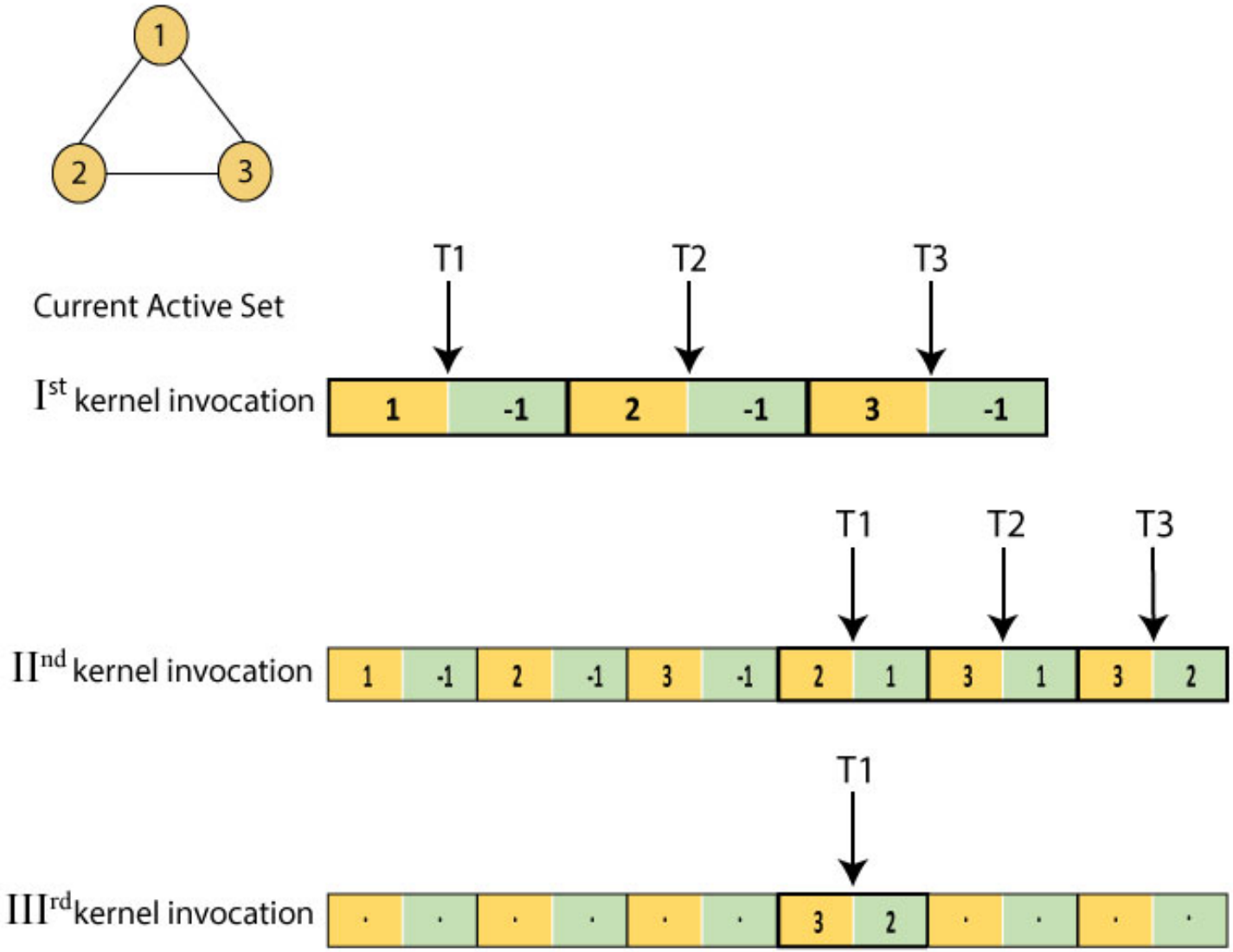
Proposed Solution

- branch and bound techniques are utilized in the design[8]
- Our method Breadth First Search for branching
- **Current Active Set**
  - The list of nodes to be branched
  - Each node in currentActiveSet becomes a BFS root
  - Each thread will take a node from currentActiveSet and evaluates a sub region of the solution space
- The branching and evaluation procedures are implemented as device functions[9]

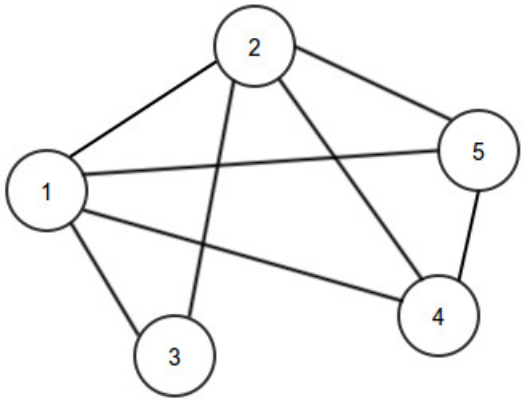
Proposed Parallel Maximal Clique Enumeration Algorithm

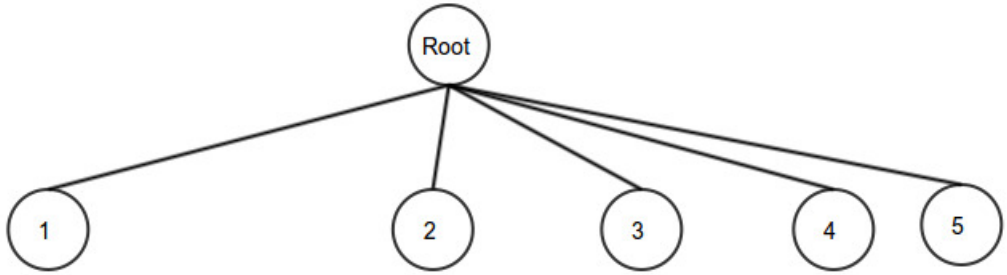
1: Read the Graph EPG
2: $v \leftarrow$ Number of vertices of Graph EPG
3: $A \leftarrow$ Initiate the *CurrentActiveSet*
4: Copy EPG into the GPU
5: **repeat**
6: $p \leftarrow$ no of nodes in A
7: **for** node i from 1 to p in **parallel do**
8: **for** all neighbour node m of i **do**
9: **if** node m is not a parent/viisted sibling of node i in A **then**
10: atomically add m to A
11: **end if**
12: **end for**
13: **end for**
14: **until** No changes in A
15: $M \leftarrow$ Backtrack A to get the Maximal Clique
16: Report M

Example

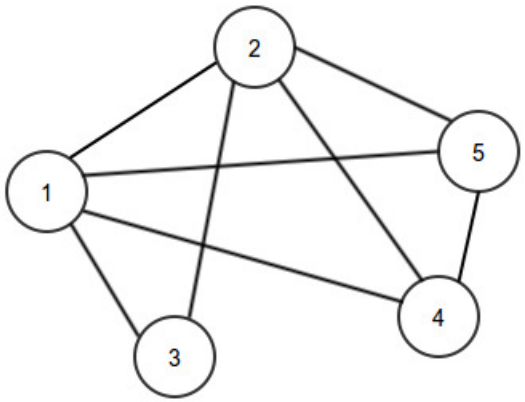

1
2
3
Current Active Set
T1
T2
T3
I^st kernel invocation
1
-1
2
-1
3
-1
T1
T2
T3
IInd kernel invocation
1
-1
2
-1
3
-1
2
1
3
1
3
2
T1
IIIrd kernel invocation
3
2

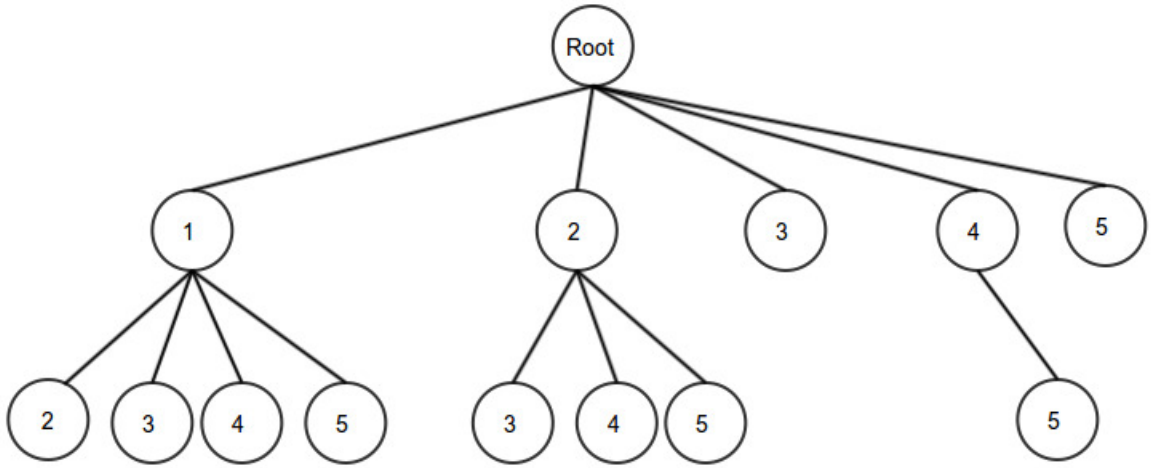
Example



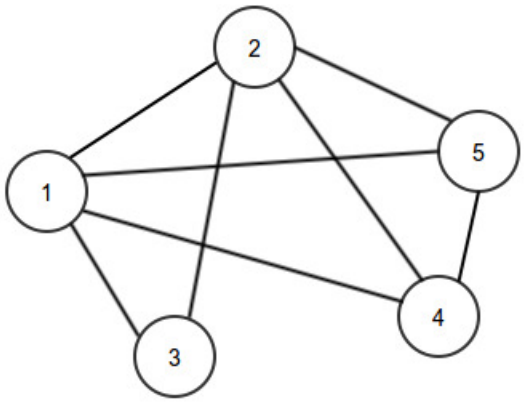


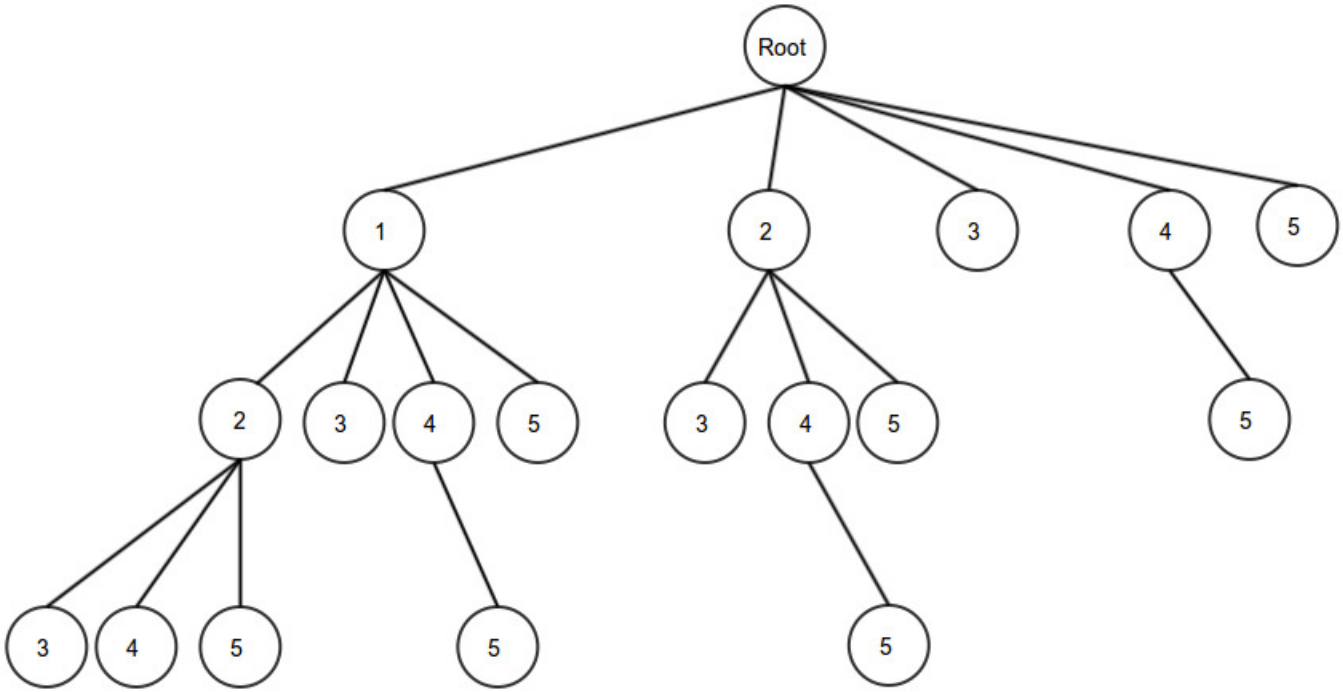
Example



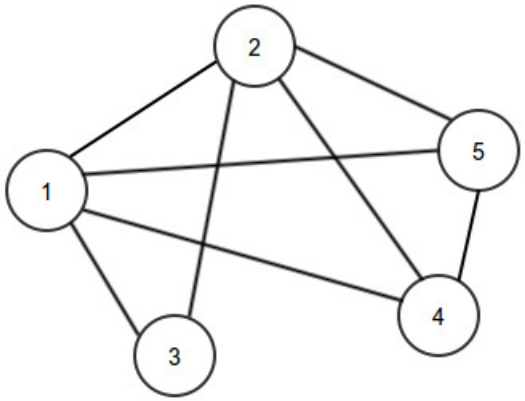


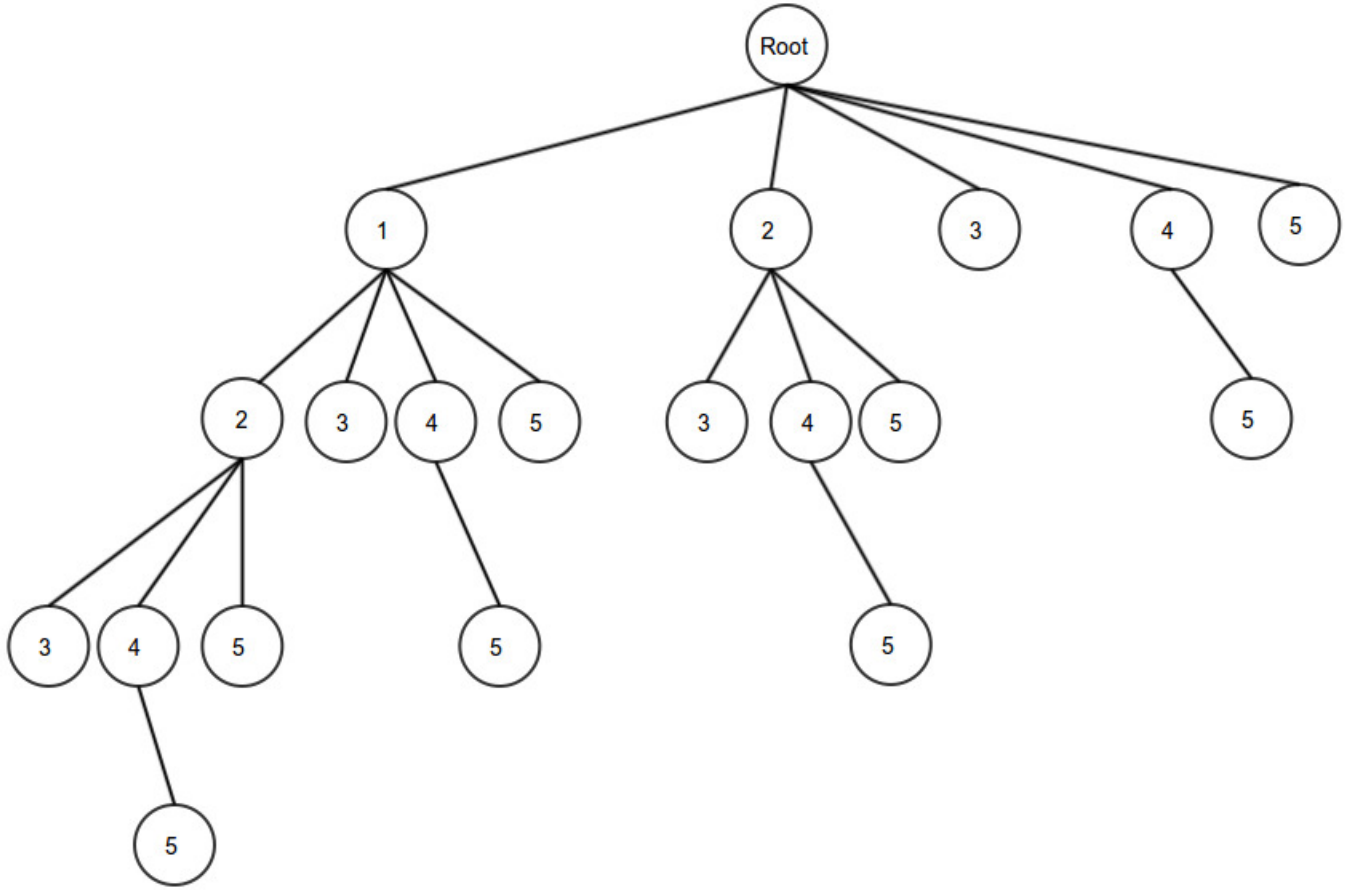
Example





Example





GPU hardware details

- GPU computing is used for parallelism [10, 11]

Table: Hardware Configuration used

Particulars	**GPU1**	**GPU2**
GPU	NVIDIA GeForce GTX 780	TESLA K20
CUDA Cores	**2304**	**2496**
GPU Clock Speed	941 MHz	706 MHz
Graphic Memory	3072 MB	4800 MB
Memory Bandwidth	288.4 GB/Sec	208 GB/Sec
Peak Performance	4 TFlops	3.52 TFlops
Compute Capability	3.5	3.5
CPU	Intel Core i7	Xeon 2650

Table: Execution times and size of maximal cliques found by the proposed parallel algorithm for different graphs of varying number of nodes.

SI. No	**Graph**	**No. of nodes in the Graph**	**Size of Max Clique**	**Exec.Time (sec) serial**	**Exec.Time (sec) GPU Parallel**
1	Graph1	50	5	0.012547	0.033104
2	Graph2	100	9	1.563211	0.297795
3	Graph3	150	10	6.43192	1.671103
4	Graph4	200	11	186.516308	4.956453
5	Graph5	250	10	2795.03886	45.19026
6	Graph6	300	11	4792.518	98.813951
7	johnson8-4-4	70	14	4.963601	1.319193
8	hamming6-4	64	4	0.002371	0.021785
9	cfat-200-1	200	12	0.179545	0.058856
10	cfat-500-1	500	14	3.32982	0.269897
11	keller4	171	11	857.2841	71.033145
12	hamming8-4	256	16	21390.449	1339.1932

Applications

The MCS problem solution has many pragmatic applications in different areas like

- Bioinformatics
- Cheminformatics
- Pattern recognition

As applications of the newly developed GPU based maximum common subgraph algorithm, two useful tools for drug discovery were developed as follows

- **LIGANDMATCHER**: A ligand matching tool to compare two ligands for their substructure similarity[12]
- **GRAPHSCREEN**: A virtual screening tool for selecting the lead molecules from a set of unknown compounds against a given active compound [13].

Application1: LIGANDMATCHER

LIGANDMATCHER

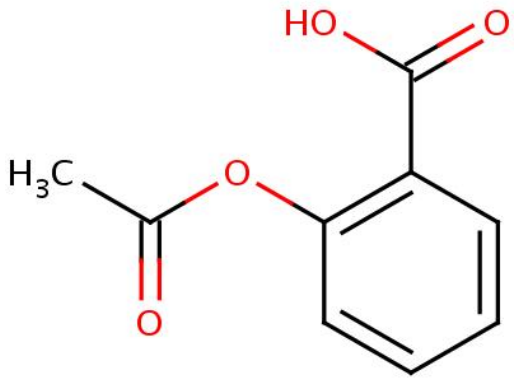


Figure: **2-acetyloxybenzoic acid**

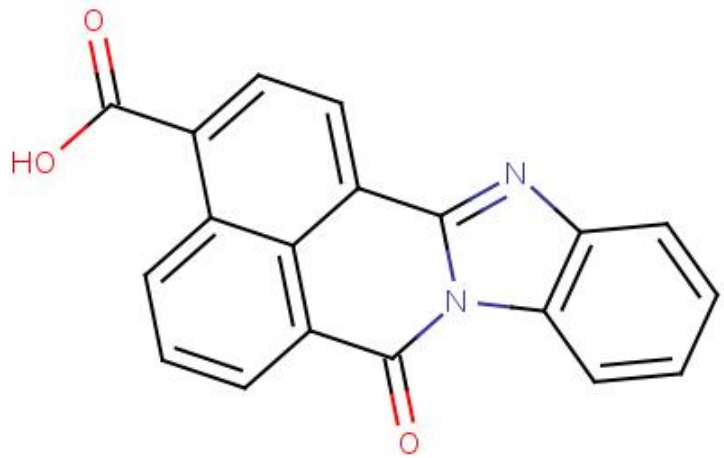


Figure:
7-oxo-7H-benzimidazo[2,1-a]benz[de]isoquinoline-3-carboxylic acid

Application1 : LIGANDMATCHER

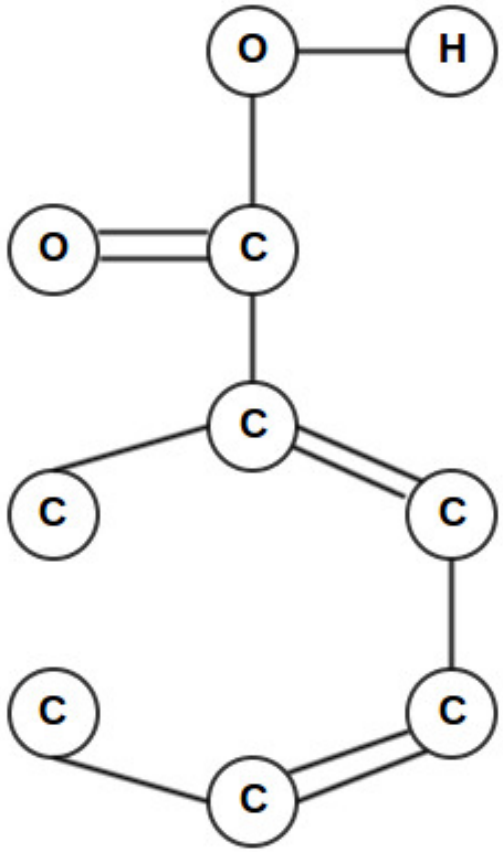


Figure: **Common substructure produced on alignment of ligands AIN and 609**

Application2 : GRAPHSCREEN

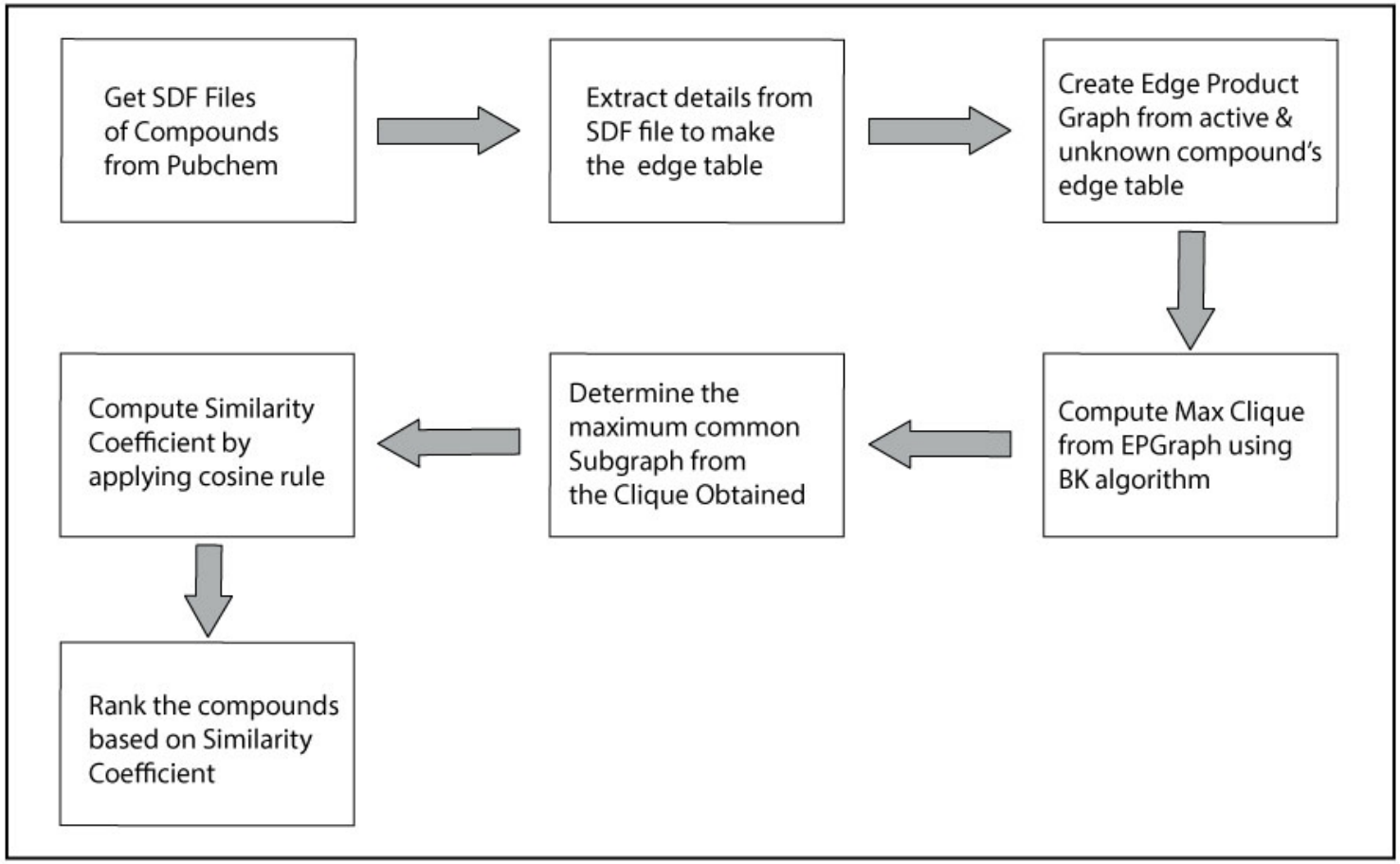


Figure: **GRAPHSCREEN - Block representation**

Application 2 : GRAPHSCREEN

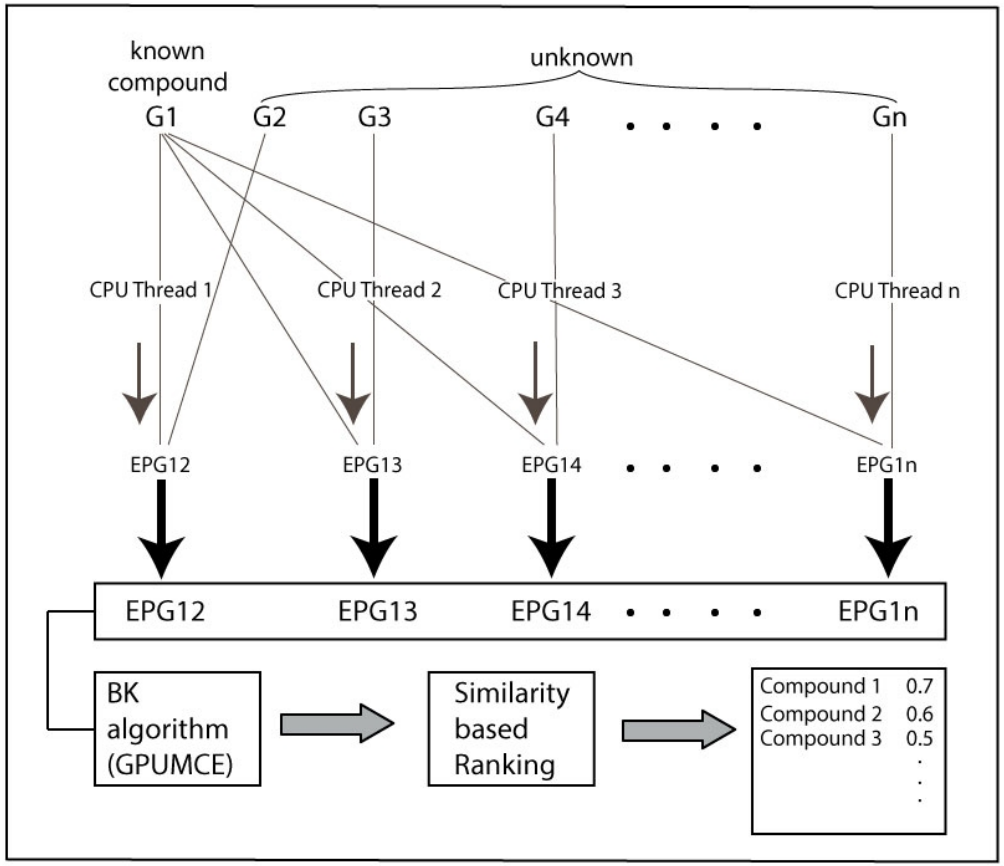


Figure: **Parallelism inGRAPHSCREEN**

Application2 : Parallel Virtual Screening Algorithm using GPUMCE

Input

ACTIVE - Structured Data File(SDF) of Active compound

UNKNOWN - Pool of SDF files of unknown compound whose ACTVITY may be predicted

Output

Ranked pool of UNKNOWN compounds by sub structure similarity

1: $GT \leftarrow$ Build Edge Table from ACTIVE input file
2: $GQ \leftarrow$ Build Edge Tables from UNKNOWN input file
3: **for** each Graph G in GQ in CPU Parallel **do**
4: $EPG \leftarrow$ BuildEdgeProductGraph(G, GT)
5: Add EPG to EPGBUFFER
6: **end for**
7: **for** each Graph G in EPGBUFFER in GPU Parallel **do**

```
8:      v ← number of vertices in G
9:      A ← Initiate the CurrentActiveSet
10:     Allocate GPU Memory
11:     Copy CurrentActiveSet into the GPU
12:     repeat
13:         p ← number of nodes in CurrentActiveSet
14:         for node i from 1 to p in parallel do
15:             for All neighbour node m of i do
16:                 if node m is not a parent of node i in A then
17:                     Atomically add m to A
18:                 end if
19:             end for
20:         end for
21:         Synchronise GPU threads
22:     until No Changes in A
23:     M ← Backtrack Active Set A to get the Maximal Clique
24:     Find the Maximum Common Subgraph from the Clique
25:     Compute the similarity of common subgraph with the query graph.
```

26: Rank the compounds according to the obtained Similarity.
27: **end for**

Table: **Comparison of CPU time and GPU time for similarity comparison for various compounds taken from PubChem[15]**

Query Compound	Unknown Compound	# nodes in EPG	Max Clique	Exec.Time (sec)serial	Exec.Time (sec)parallel
(18,19)	(23,24)	89	15	356.70	5.33
(18,19)	(22,23)	103	15	924.26	4.56
(18,19)	(20,21)	85	13	210.89	3.09
(18,19)	(35,37)	136	12	*	3.69
(18,19)	(38,40)	148	10	*	4.56
(18,19)	(17,17)	53	10	93.40	3.08
(18,19)	(31,32)	109	7	1028.35	3.27
(20,21)	(31,32)	133	15	*	5.68
(20,21)	(22,23)	125	10	*	5.63
(25,26)	(23,24)	155	16	*	17.4
(25,26)	(20,21)	144	14	*	5.49
(31,31)	(17,17)	87	10	346.08	3.30

* - Not Executed due to serial exception error

Table: **First 10 ranked elements by the proposed algorithm when Benzene was used as query compound for virtual screening**

Rank	Query	Unknown	No.of bonds in Query	No.of bonds in Unknown	Cosine Similarity
1	Benzene	compound-94089	13	11	0.585369
2	Benzene	compound-2436	13	12	0.560449
3	Benzene	compound-32616	13	12	0.560449
4	Benzene	compound-39287	13	12	0.560449
5	Benzene	compound-55925	13	12	0.560449
6	Benzene	compound-843	13	13	0.538462
7	Benzene	compound-11906	13	13	0.538462
8	Benzene	compound-16672	13	13	0.538462
9	Benzene	compound-19285	13	13	0.538462
10	Benzene	compound-19487	13	13	0.526235

Conclusion

- The technique to represent compounds as graphs and finding maximum common subgraph problem has pragmatic applications in various areas like bioinformatics, chemistry, pattern recognition etc.
- As the size of graph surges the solution search space also grows combinatorially. Therefore parallel solutions are favored over serial solutions of MCS. We implemented the Bron-Kerbosch algorithm using parallel computing solution and reduced the running time significantly.
- The GPU based MCS is a viable alternative to quickly compare graphs at a comparitively lower cost.
- As part of the work, two useful tools(LIGAANDMATCHER GRAPHSCREEN) for Drug Discovery was developed

Availability Requirments

- Name of the tool: GRAPHSCREEN, LIGANDMATCHER
- Source code: http://ccc.nitc.ac.in/project/GPUMCS/
- Operating System Used: Linux Ubuntu 13.10
- Programming Language Used: C, Python
- Frame Work Used: CUDA, PyCUDA

THANKS

QUESTIONS!!

References I

[1] Ina Koch.
Enumerating all connected maximal common subgraphs in two graphs.
Theoretical Computer Science, 250(1-2):1–30, 2001.

[2] H. Bunke, P.Foggia, C. Guidobaldi, C. Sansone, and M. Vento.
A comparison of algorithms for maximal common subgraph on randomly connected graphs.
Structural, Syntactic and Statistical pattern recongnition, pages 85–106, 2002.

[3] Ullmann and Julian R.
An algorithm for subgraph isomorphism.
Journal of the ACM, 23:31–42, 1976.

References II

[4] James J. McGregor.
Backtrack search algorithms and maximal common subgraph problem.
Software-Practice and Experience, 12:23–34, 1982.

[5] G. Levi.
A note on the derivation of maximal common subgraphs of two directed or undirected graphs.
Calcolo, 9:341–352, 1972.

[6] Coen Bron and Joep Kerbosch.
Algorithm 457: finding all cliques of an undirected graph.
Commun. ACM, 16(9):575–577, September 1973.

References III

[7] Matthew C. Schmidt, Nagiza F. Samatova, Kevin thomas, and Byung-Hoon Park.
A scalable, parallel algorithm for maximal clique enumeration.
Journal of Parallel distributed computing, 69:417–428, 2009.

[8] T Carnetro, M Muritiba, A. E De Campos, and G. A. L Negreiros.
Solving atsp hard instances by new parallel branch and bound algorithm using gpgp.
Iberian Latin American Congress on Computational Methods in Engineering (CILAMCE), 2011.

References IV

[9] Krzysztof Kurowski and Mikolaj Mackowiak.
Parallel branch and bound method for solving traveling salesman problem using cpu and gpgpu volunteer computing and the xmpp protocol.
The 25th International Conference on Advanced Information Networking and ApplicationsBiopolis, Singapore, March 22-25, 2011.

[10] Jie Cheng.
CUDA by Example: An Introduction to General-Purpose GPU Programming.
Addison-Wesley, 2010.

[11] David B. Kirk and Wen mei. W. Hwu.
Programming massively parallel processors, A hands on approach.
Morgan Kaufmann, 2010.

References V

[12] Yiqun Cao, Tao Jiang, and Thomas Girke.
A maximum common substructure-based algorithm for searching and predicting drug-like compounds.
BIOINFORMATICS, 24 ISMB:366 374, 2008.

[13] A. C. Schierz.
Virtual screening of bioassay data.
Journal of Cheminformatics, 21:1–12, December 2009.

[14] RCSB protein data bank.
`http://www.rcsb.org/pdb/home/home.do`.

[15] NCBI PubChem, 2008.
[Online; accessed 29-April-2014].